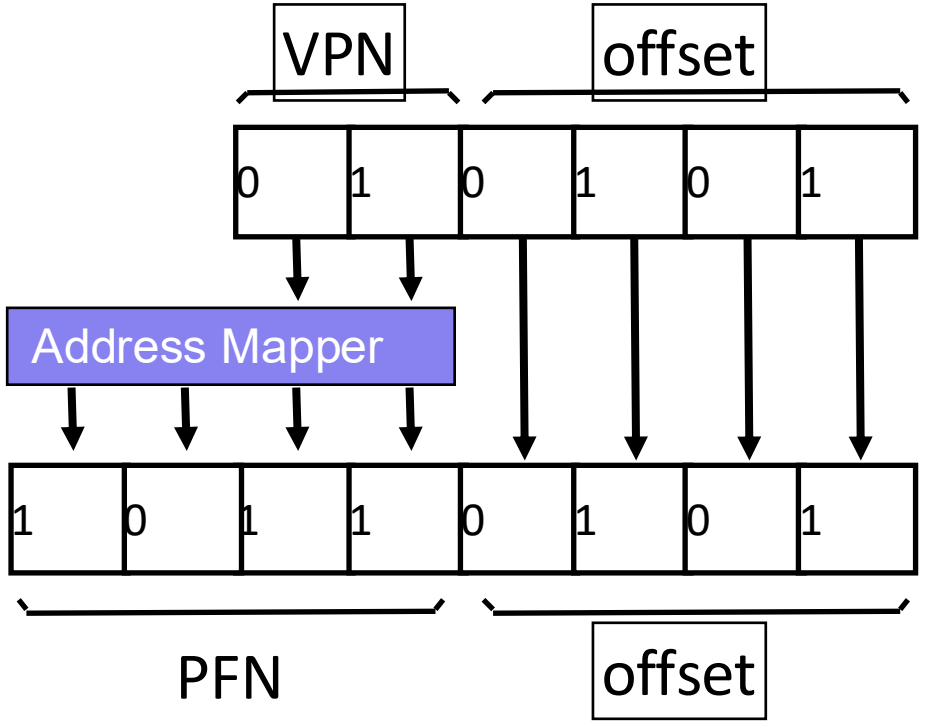
Memory Virtualization



Review: Virt ➔ Phys page mapping

Number of bits in the virtual address not equal the number of bits in the physical address



How should OS translate VPN to PFN?

For segmentation, OS used a formula (e.g., phys addr = virt_offset + base_reg)

For paging, OS needs a different mapping mechanism

Q: What data structure is good?

Linear Page Table

A simple option: an array

Review: Avoid the simple linear page table

Use more efficient (but complex) data structures, instead of the simple big array

Any data structure is possible in principle!

The hardware needs to know how to access it

… or a “cache” of it (we’ll see about TLBs later)

Other Approaches

1. Inverted Pagetables
2. Segmented Pagetables
3. Multi-level Pagetables

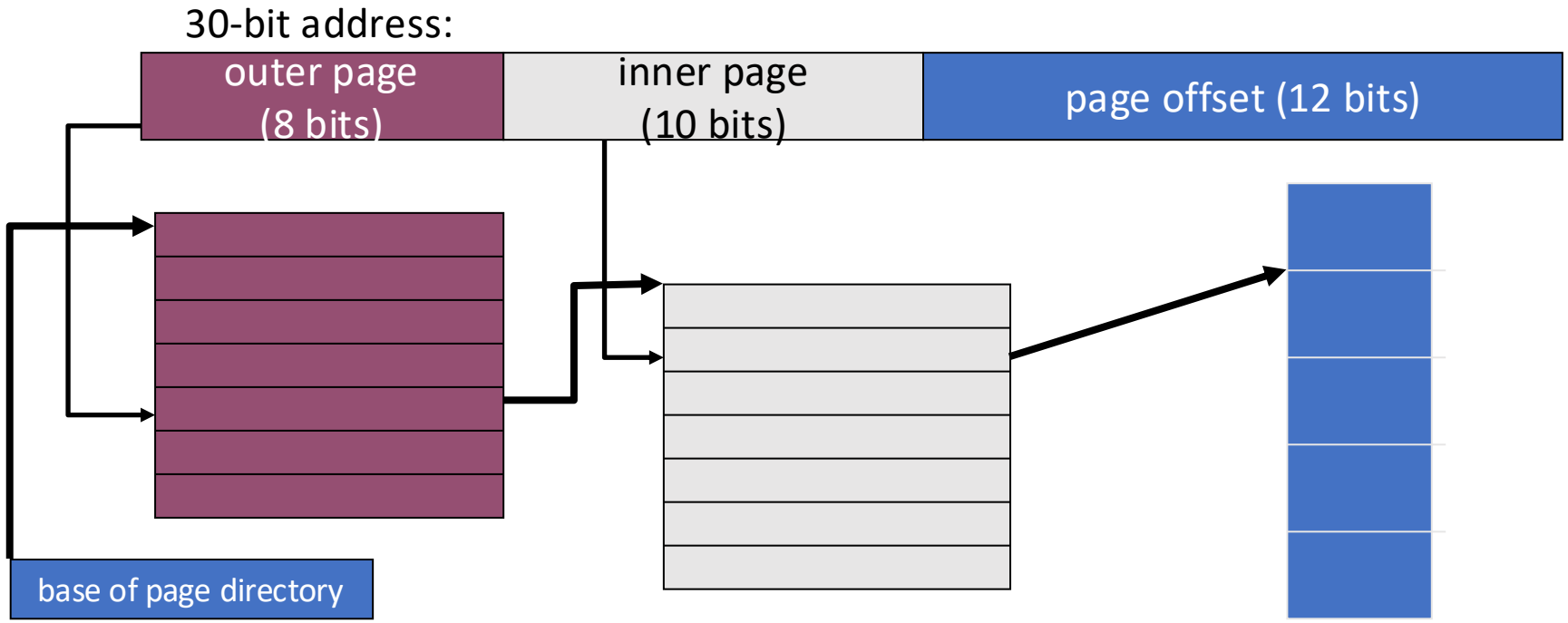
"Paging the page tables"

3) Multilevel Page Tables

Goal: Allow page tables to be allocated non-contiguously

Idea: Page the page tables

- Creates multiple levels of page tables; outer level page directory
- Only allocate page tables for pages in use
- hardware can walk known structure; e.g. x86



Multilevel example

page directory

VPN	PPN	valid
0	0x3	1
1	-	0
2	-	0
-	-	0
-	-	0
-	-	0
-	-	0
-	-	0
-	-	0
-	-	0
-	-	0
-	-	0
-	-	0
-	-	0
-	-	0
-	-	0
15	0x92	1

page of PT (@PPN:0x3)

PPN	valid
0x10	1
0x23	1
-	0
-	0
0x80	1
0x59	1
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0

page of PT (@PPN:0x92)

PPN	valid
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0
0x55	1
0x45	1

translate 0x01ABC
0x23ABC

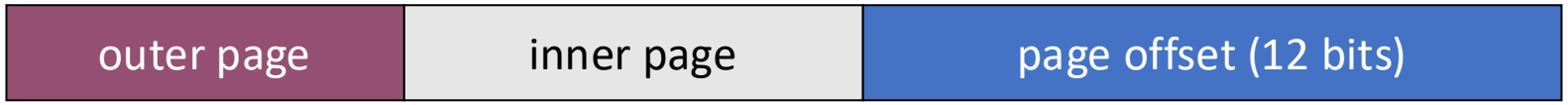
translate 0x00000
0x10000

translate 0xFEED0
0x55ED0

20-bit address:



Address format for Multilevel Paging



How should virtual address be structured?

- How many bits for each paging level?

Goal?

- Page table at each level fits within a page
- PTE size * number PTE = page size
  - Assume PTE size = 4 bytes
  - Page size = 2^12 bytes = 4KB
  - number PTE per page = (2^12 bytes per page) / (4 bytes per PTE)
  - → number PTE = 2^10
- → # bits for selecting inner page = 10

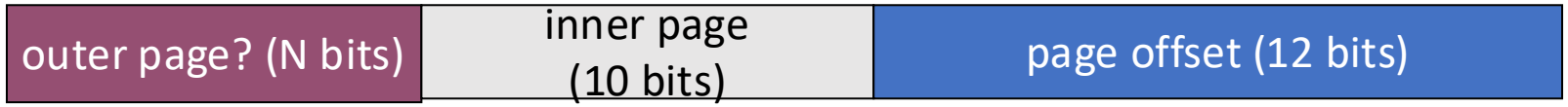
Remaining bits for outer page:

- 30 – 10 – 12 = 8 bits

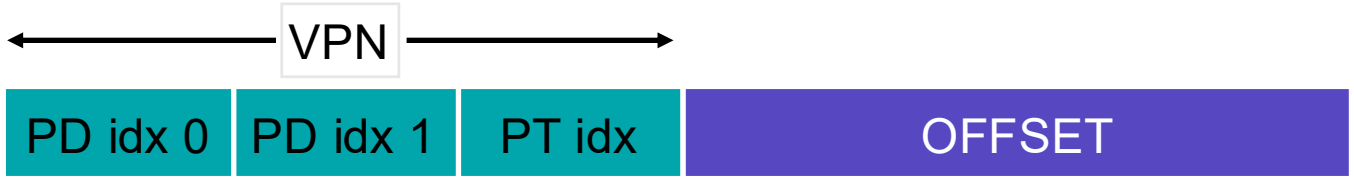
Problem with 2 levels?

Problem: page directory (outer level) may not fit in a page!

Solution:



- Split page directories into pieces
- Use another page dir to refer to the pieces of the page directory



How large is virtual address space with 4 KB pages, 4 byte PTEs, each page table fits in page, given 1, 2, 3 levels?

4KB / 4 bytes → 1K entries per level

1 level: 1K * 4K = **2^22** = 4 MB

2 levels: 1K * 1K * 4K = **2^32** ≈ 4 GB

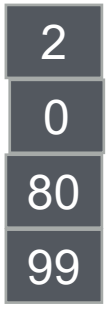
3 levels: 1K * 1K * 1K * 4K = **2^42** ≈ 4 TB

Memory Accesses with Pages

```
0x0010:  movl   0x1100, %edi
0x0013:  addl $0x3, %edi
0x0019:  movl   %edi, 0x1100
```

Assume 14-bit virtual address
Assume PTE's are 4 bytes
Assume 4KB pages
How many bits for offset?
Assume a 3-level page table

Conceptual linear view of page table



Earlier: How many mem refs with linear page table?

10 (3 inst + 2 load/store + 5 page table)

Physical Memory Accesses with Paging?

1) Fetch instruction at virtual addr 0x0010; vpn?

- Access page table to get pfn for vpn 0
- Mem ref 1,2,3: get pfn
- Learn vpn 0 is at pfn 2
- Fetch instruction at 0x2010 (Mem ref 4)

Exec, load from virtual addr 0x1100; vpn?

- Access page table to get pfn for vpn 1
- Mem ref 5,6,7: get pfn
- Learn vpn 1 is at pfn 0
- Movl from 0x0100 into reg (Mem ref 8)

Use of a k-level page table results in (k+1) times more memory refs

Review: Paging pros and cons

Advantages

- No external fragmentation
  - don’t need to find contiguous RAM
- All free pages are equivalent
  - Easy to manage, allocate, and free pages

Disadvantages

- Page tables can get big
  - Must have one entry for every page of address space
- Accessing page tables is too slow [address this next]
  - Multiplies the number of memory references per instruction

Translation Lookaside Buffers

How can page translations be made faster?

What is the basic idea of a TLB (Translation Lookaside Buffer)?

What types of workloads perform well with TLBs?

How do TLBs interact with context-switches?

Example: Array Iterator

```
int sum = 0;
for (i=0; i<N; i++){
      sum += a[i];
}
```

Assume ‘a’ starts at 0x3000

PTE at 0x100C; PFN 0x7

Linear page table

Ignore instruction fetches

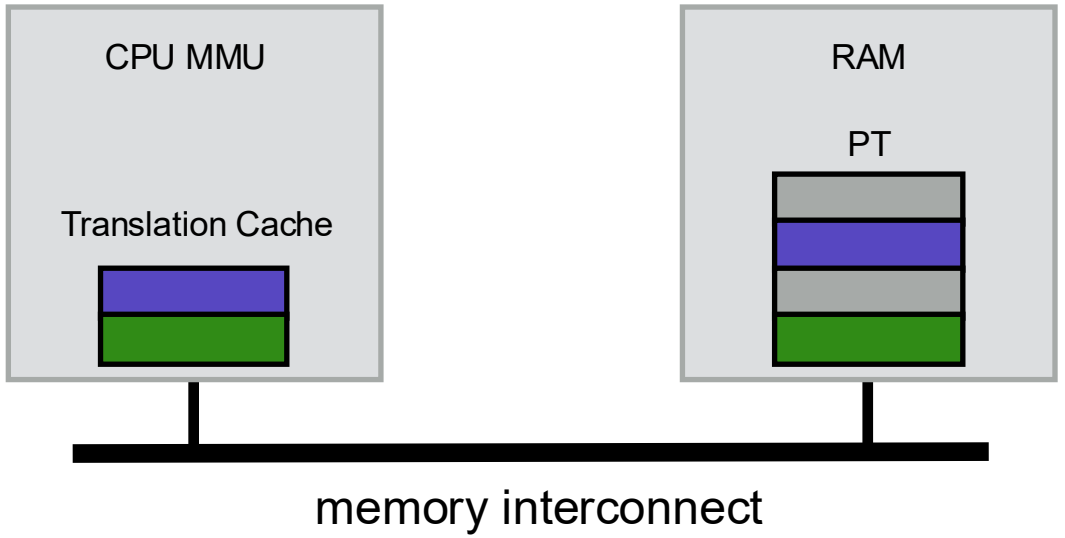
Assume 4 KB pages

What virtual addresses?	What physical addresses?
load 0x3000	load 0x100C
	load 0x7000
load 0x3004	load 0x100C
	load 0x7004
load 0x3008	load 0x100C
	load 0x7008
load 0x300C	load 0x100C
	load 0x700C
...	

Observation:
Repeatedly access same PTE because program repeatedly accesses same virtual page

Strategy: Cache Page Translations

We couldn't store entire page table in MMU, but we can store a fast cache



Some popular entries

TLB: **T**ranslation **L**ookaside **B**uffer

TLB Control Flow

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
```

TLB Control Flow

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)   // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset   = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
```

TLB Control Flow

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)   // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset   = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else                   // TLB Miss
```

TLB Control Flow

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)   // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset   = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else                    // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE))  // assume simple linear page table
12     PTE = AccessMemory(PTEAddr)
```

TLB Control Flow

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)   // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset   = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else                    // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE))  // assume simple linear page table
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
```

TLB Control Flow

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)   // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset   = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else                    // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE))  // assume simple linear page table
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19         RetryInstruction()
```